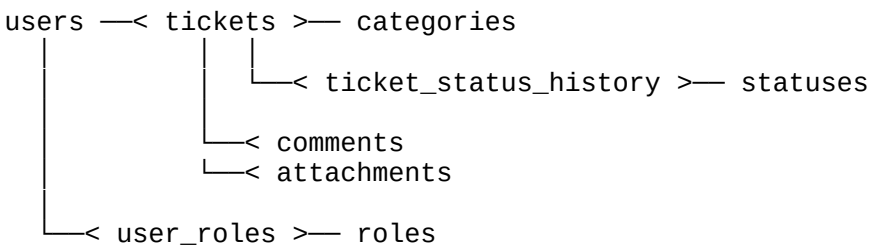
ER-модель ServiceDesk (SQLite)

1. Сущности и связи

text



2. Таблицы, ключи, ограничения

Таблица	PK	FK	Ключевые ограничения
roles	id	—	`name UNIQUE NOT NULL`
users	id	—	`email UNIQUE NOT NULL` (с CHECK на формат), `is_active DEFAULT 1`
user_roles	(user_id, role_id)	user_id → users, role_id → roles	составной PK, `ON DELETE CASCADE`
categories	id	—	`name UNIQUE NOT NULL`, `parent_id→categories(id)` (иерархия, `ON DELETE SET NULL`)
statuses	id	—	`name UNIQUE NOT NULL`, `is_closed DEFAULT 0`
tickets	id	author_id → users, assignee_id → users (nullable), category_id → categories, status_id → statuses	`title NOT NULL`, `priority CHECK IN ('low','normal','high','critical')`, `created_at DEFAULT CURRENT_TIMESTAMP`
comments	id	ticket_id → tickets, author_id → users	`body NOT NULL`, `ON DELETE CASCADE` по ticket
attachments	id	ticket_id → tickets, comment_id → comments (nullable), uploaded_by → users	CHECK: хотя бы один из ticket_id/comment_id задан
ticket_status_history	id	ticket_id → tickets, status_id → statuses, changed_by → users	для аудита смены статусов

Замечания:

- SQLite не имеет нативных ENUM — используем `CHECK`.
- Внешние ключи включаются через `PRAGMA foreign_keys = ON`.
- Для часто используемых FK создаём индексы.

- Полнотекстовый поиск по `title`/`body` — через `FTS5` (опционально, показано ниже).

3. `schema.sql`

sql

```
PRAGMA foreign_keys = ON;
PRAGMA journal_mode = WAL;

-- ---------- roles ----------
CREATE TABLE roles (
    id   INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE,
    description TEXT
);

-- ---------- users ----------
CREATE TABLE users (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    email       TEXT NOT NULL UNIQUE CHECK (email LIKE '%_@_%._%'),
    full_name   TEXT NOT NULL,
    department  TEXT,
    is_active   INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)),
    created_at  TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- ---------- user_roles (M:N) ----------
CREATE TABLE user_roles (
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role_id INTEGER NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, role_id)
);

-- ---------- categories (self-ref) ----------
CREATE TABLE categories (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    name      TEXT NOT NULL UNIQUE,
    parent_id INTEGER REFERENCES categories(id) ON DELETE SET NULL
);

-- ---------- statuses ----------
CREATE TABLE statuses (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    name      TEXT NOT NULL UNIQUE,
    is_closed INTEGER NOT NULL DEFAULT 0 CHECK (is_closed IN (0,1))
);

-- ---------- tickets ----------
CREATE TABLE tickets (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    title        TEXT NOT NULL,
    description  TEXT,
    author_id    INTEGER NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    assignee_id  INTEGER          REFERENCES users(id) ON DELETE SET NULL,
    category_id  INTEGER          REFERENCES categories(id) ON DELETE SET NULL,
    status_id    INTEGER NOT NULL REFERENCES statuses(id) ON DELETE RESTRICT,
    priority     TEXT NOT NULL DEFAULT 'normal'
                 CHECK (priority IN ('low','normal','high','critical')),
    created_at   TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at   TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    closed_at    TEXT
);
```

```
CREATE INDEX idx_tickets_author    ON tickets(author_id);
CREATE INDEX idx_tickets_assignee  ON tickets(assignee_id);
CREATE INDEX idx_tickets_status    ON tickets(status_id);
CREATE INDEX idx_tickets_category  ON tickets(category_id);
CREATE INDEX idx_tickets_created   ON tickets(created_at);

-- ---------- comments ----------
CREATE TABLE comments (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    ticket_id  INTEGER NOT NULL REFERENCES tickets(id) ON DELETE CASCADE,
    author_id  INTEGER NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    body       TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_internal INTEGER NOT NULL DEFAULT 0 CHECK (is_internal IN (0,1))
);

CREATE INDEX idx_comments_ticket ON comments(ticket_id);

-- ---------- attachments ----------
CREATE TABLE attachments (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    ticket_id   INTEGER          REFERENCES tickets(id)  ON DELETE CASCADE,
    comment_id  INTEGER          REFERENCES comments(id) ON DELETE CASCADE,
    uploaded_by INTEGER NOT NULL REFERENCES users(id)    ON DELETE RESTRICT,
    file_name   TEXT NOT NULL,
    file_size   INTEGER NOT NULL CHECK (file_size >= 0),
    mime_type   TEXT,
    uploaded_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (ticket_id IS NOT NULL OR comment_id IS NOT NULL)
);

CREATE INDEX idx_attach_ticket  ON attachments(ticket_id);
CREATE INDEX idx_attach_comment ON attachments(comment_id);

-- ---------- ticket_status_history ----------
CREATE TABLE ticket_status_history (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    ticket_id  INTEGER NOT NULL REFERENCES tickets(id)  ON DELETE CASCADE,
    status_id  INTEGER NOT NULL REFERENCES statuses(id) ON DELETE RESTRICT,
    changed_by INTEGER NOT NULL REFERENCES users(id)    ON DELETE RESTRICT,
    changed_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    note       TEXT
);

CREATE INDEX idx_hist_ticket ON ticket_status_history(ticket_id);

-- ---------- Триггер: обновление updated_at и closed_at ----------
CREATE TRIGGER trg_tickets_updated_at
AFTER UPDATE OF status_id ON tickets
FOR EACH ROW
BEGIN
    UPDATE tickets
       SET updated_at = CURRENT_TIMESTAMP,
           closed_at  = CASE
                            WHEN (SELECT is_closed FROM statuses WHERE id =
NEW.status_id) = 1
                                 AND NEW.closed_at IS NULL THEN
CURRENT_TIMESTAMP
                            WHEN (SELECT is_closed FROM statuses WHERE id =
NEW.status_id) = 0
                                 THEN NULL
                            ELSE NEW.closed_at
                        END
```

```
    WHERE id = NEW.id;
END;
```

4. Тестовые данные (`seed.sql`)

sql

```
INSERT INTO roles (name, description) VALUES
 ('admin',    'Администратор системы'),
 ('agent',    'Агент поддержки'),
 ('customer', 'Конечный пользователь');

INSERT INTO statuses (name, is_closed) VALUES
 ('new',       0),
 ('in_progress',0),
 ('waiting',   0),
 ('resolved',  1),
 ('closed',    1);

INSERT INTO categories (name, parent_id) VALUES
 ('IT',        NULL),
 ('Hardware',  1),
 ('Software',  1),
 ('HR',        NULL),
 ('Access',    NULL);

INSERT INTO users (email, full_name, department, is_active) VALUES
 ('admin@sd.local',   'Анна Администова',  'IT',        1),
 ('agent1@sd.local',  'Игорь Агентов',  'Support',   1),
 ('agent2@sd.local',  'Ольга Агентова', 'Support',   1),
 ('user1@sd.local',   'Пётр Пользователев','Sales',  1),
 ('user2@sd.local',   'Мария Клиентова','Marketing', 1),
 ('user3@sd.local',   'Сергей Ушедший', 'Sales',     0);

INSERT INTO user_roles (user_id, role_id) VALUES
 (1,1),(2,2),(3,2),(4,3),(5,3),(6,3);

INSERT INTO tickets (title, description, author_id, assignee_id, category_id,
status_id, priority, created_at) VALUES
 ('Не включается ноутбук',        'При нажатии кнопки нет реакции', 4, 2, 2, 2,
'high',     '2024-03-01 09:15:00'),
 ('Ошибка Outlook 0x800ccc0e',    'Не отправляются письма',         5, 3, 3, 2,
'normal',   '2024-03-02 10:20:00'),
 ('Нужен доступ к 1С',            'Прошу выдать доступ',            4, 2, 5, 1,
'normal',   '2024-03-03 11:00:00'),
 ('Сбой VPN',                     'Постоянные разрывы соединения',  5, 3, 1, 4,
'high',     '2024-03-04 14:30:00'),
 ('Замена картриджа',             'Принтер HP в 3-м кабинете',      4, NULL,2,
1, 'low',     '2024-03-05 08:00:00');

INSERT INTO comments (ticket_id, author_id, body, created_at, is_internal)
VALUES
 (1, 2, 'Уточните модель ноутбука', '2024-03-01 09:30:00', 0),
 (1, 4, 'Dell Latitude 5420',        '2024-03-01 09:35:00', 0),
 (1, 2, 'Проверяем блок питания',    '2024-03-01 10:00:00', 1),
 (2, 3, 'Пересоздайте профиль',      '2024-03-02 11:00:00', 0),
 (4, 3, 'Обновлён клиент VPN',       '2024-03-04 16:00:00', 0);

INSERT INTO attachments (ticket_id, comment_id, uploaded_by, file_name,
file_size, mime_type) VALUES
 (1, NULL, 4, 'photo1.jpg', 204800, 'image/jpeg'),
 (1, NULL, 4, 'photo2.jpg', 180224, 'image/jpeg'),
```

```
 (2, 4,    3, 'log.txt',    1024,   'text/plain');

INSERT INTO ticket_status_history (ticket_id, status_id, changed_by, note)
VALUES
 (1, 1, 4, 'Создана пользователем'),
 (1, 2, 2, 'Принята в работу'),
 (4, 1, 5, 'Создана пользователем'),
 (4, 2, 3, 'Взята в работу'),
 (4, 4, 3, 'Решено');
```

5. Сложные SQL-запросы

5.1 Полная карточка заявки со всеми связанными данными

sql

```
SELECT
    t.id,
    t.title,
    t.priority,
    s.name            AS status,
    c.name            AS category,
    a.full_name       AS author,
    asg.full_name     AS assignee,
    (SELECT COUNT(*) FROM comments    cm WHERE cm.ticket_id = t.id) AS
comments_cnt,
    (SELECT COUNT(*) FROM attachments at WHERE at.ticket_id = t.id) AS
attachments_cnt
FROM tickets t
JOIN statuses s   ON s.id = t.status_id
LEFT JOIN categories c ON c.id = t.category_id
JOIN users a      ON a.id = t.author_id
LEFT JOIN users asg ON asg.id = t.assignee_id
ORDER BY t.created_at DESC;
```

5.2 Заявки с последним комментарием и его автором (коррелированный подзапрос + JOIN)

sql

```
SELECT
    t.id,
    t.title,
    cm.body          AS last_comment,
    u.full_name      AS last_comment_author,
    cm.created_at    AS last_comment_at
FROM tickets t
LEFT JOIN comments cm
       ON cm.id = (
            SELECT id FROM comments
             WHERE ticket_id = t.id
             ORDER BY created_at DESC
             LIMIT 1
       )
LEFT JOIN users u ON u.id = cm.author_id
ORDER BY t.id;
```

5.3 Топ-5 агентов по закрытым заявкам (CTE + JOIN + GROUP BY)

sql

```
WITH closed_ids AS (
    SELECT id FROM statuses WHERE is_closed = 1
)
SELECT
    u.id,
    u.full_name,
    COUNT(*)                                    AS closed_tickets,
    ROUND(AVG(julianday(t.closed_at) - julianday(t.created_at)), 2) AS
avg_days_to_close
FROM tickets t
JOIN users u       ON u.id = t.assignee_id
JOIN closed_ids ci ON ci.id = t.status_id
GROUP BY u.id, u.full_name
ORDER BY closed_tickets DESC, avg_days_to_close ASC
LIMIT 5;
```

5.4 Заявки без назначенного исполнителя старше 3 дней (просрочка SLA)

sql

```
SELECT t.id, t.title, t.priority, t.created_at,
       CAST(julianday('now') - julianday(t.created_at) AS INTEGER) AS age_days
FROM tickets t
JOIN statuses s ON s.id = t.status_id
WHERE t.assignee_id IS NULL
  AND s.is_closed = 0
  AND julianday('now') - julianday(t.created_at) > 3
ORDER BY age_days DESC;
```

5.5 Полная история смены статусов с временем пребывания в каждом статусе (оконные функции)

sql

```
SELECT
    t.id AS ticket_id,
    t.title,
    sh.changed_at,
    st.name                AS status,
    u.full_name            AS changed_by,
    LEAD(sh.changed_at) OVER (PARTITION BY sh.ticket_id ORDER BY sh.changed_at)
AS next_change,
    ROUND(
        julianday(LEAD(sh.changed_at) OVER (PARTITION BY sh.ticket_id ORDER BY
sh.changed_at))
        - julianday(sh.changed_at), 2
    ) AS hours_in_status
FROM ticket_status_history sh
JOIN tickets  t ON t.id = sh.ticket_id
JOIN statuses st ON st.id = sh.status_id
JOIN users    u ON u.id = sh.changed_by
ORDER BY sh.ticket_id, sh.changed_at;
```

5.6 Рекурсивный CTE: дерево категорий с числом заявок

sql

```
WITH RECURSIVE tree(id, name, parent_id, path, depth) AS (
    SELECT id, name, parent_id, name, 0
      FROM categories
     WHERE parent_id IS NULL
    UNION ALL
    SELECT c.id, c.name, c.parent_id, tree.path || ' / ' || c.name, tree.depth +
1
      FROM categories c
      JOIN tree ON c.parent_id = tree.id
)
SELECT
    tree.id,
    tree.path                AS category_path,
    tree.depth,
    COUNT(t.id)              AS tickets_count
FROM tree
LEFT JOIN tickets t ON t.category_id = tree.id
GROUP BY tree.id, tree.path, tree.depth
ORDER BY tree.path;
```

5.7 Поиск заявок по слову в заголовке/описании/комментариях (UNION + EXISTS)

sql

```
SELECT DISTINCT t.id, t.title, t.created_at
FROM tickets t
WHERE t.title       LIKE '%VPN%'
   OR t.description LIKE '%VPN%'
   OR EXISTS (
        SELECT 1 FROM comments c
         WHERE c.ticket_id = t.id
           AND c.body LIKE '%VPN%'
      )
ORDER BY t.created_at DESC;
```

5.8 Дашборд: сводка по статусам и приоритетам (pivot через CASE)

sql

```
SELECT
    s.name AS status,
    SUM(CASE WHEN t.priority = 'low'      THEN 1 ELSE 0 END) AS low_cnt,
    SUM(CASE WHEN t.priority = 'normal'   THEN 1 ELSE 0 END) AS normal_cnt,
    SUM(CASE WHEN t.priority = 'high'     THEN 1 ELSE 0 END) AS high_cnt,
    SUM(CASE WHEN t.priority = 'critical' THEN 1 ELSE 0 END) AS critical_cnt,
    COUNT(t.id) AS total
FROM statuses s
LEFT JOIN tickets t ON t.status_id = s.id
GROUP BY s.id, s.name
ORDER BY s.id;
```

5.9 Пользователи без единой заявки (LEFT JOIN + IS NULL / NOT EXISTS)

sql

```
SELECT u.id, u.email, u.full_name
FROM users u
WHERE NOT EXISTS (SELECT 1 FROM tickets t WHERE t.author_id = u.id)
  AND NOT EXISTS (SELECT 1 FROM tickets t WHERE t.assignee_id = u.id)
  AND u.is_active = 1;
```

5.10 Заявки, у которых исполнитель — тот же, кто и автор (аномалия для аудита)

sql

```
SELECT t.id, t.title, t.author_id, t.assignee_id
FROM tickets t
WHERE t.author_id = t.assignee_id;
```

6. Дополнительно: FTS5 для полнотекстового поиска

Если нужен быстрый поиск — добавьте виртуальную таблицу:

sql

```
CREATE VIRTUAL TABLE tickets_fts USING fts5(
    title, description,
    content='tickets', content_rowid='id'
);

-- триггеры синхронизации
CREATE TRIGGER tickets_ai AFTER INSERT ON tickets BEGIN
  INSERT INTO tickets_fts(rowid, title, description)
  VALUES (new.id, new.title, new.description);
END;

CREATE TRIGGER tickets_ad AFTER DELETE ON tickets BEGIN
  INSERT INTO tickets_fts(tickets_fts, rowid, title, description)
  VALUES('delete', old.id, old.title, old.description);
END;

CREATE TRIGGER tickets_au AFTER UPDATE ON tickets BEGIN
  INSERT INTO tickets_fts(tickets_fts, rowid, title, description)
  VALUES('delete', old.id, old.title, old.description);
  INSERT INTO tickets_fts(rowid, title, description)
  VALUES (new.id, new.title, new.description);
END;

-- запрос
SELECT t.*, bm25(tickets_fts) AS score
FROM tickets_fts
JOIN tickets t ON t.id = tickets_fts.rowid
WHERE tickets_fts MATCH 'VPN OR Outlook'
ORDER BY score;
```

Резюме архитектурных решений

- **Нормализация до 3НФ**, справочники вынесены (`statuses`, `roles`, `categories`).
- **M:N** между пользователями и ролями через `user_roles`.
- **Иерархия категорий** через self-reference + рекурсивный CTE.
- **Аудит** изменений статусов — отдельная таблица `ticket_status_history`.
- **Ограничения целостности** заданы на уровне БД (`CHECK`, `FK` с `ON DELETE`), а не только в коде.
- **Индексы** на все FK и `created_at` для типовых отчётов.

- **Триггер** автоматически поддерживает `updated_at`/`closed_at`.
- **FTS5** — опциональный быстрый поиск без внешних зависимостей.